# KeelBase — 业务安全的 AI 应用工程平台

> **构建企业可信任的 AI 应用——从零开始，或从既有系统出发。默认受治理、可审计、可私有化部署。**
>
> Open-source Business-safe AI Runtime：让 AI Agent 接入既有业务系统，在权限、确认、审计、撤销的边界内真正完成业务工作，**不替换既有技术栈。**
>
> 能力边界与撤销档位见 SECURITY.md Trust Boundaries——审计链为**应用边界内篡改即断链**；撤销仅限本系统创建的数据，外部系统副作用如实呈现补偿状态。

---

## 1. 项目简介

KeelBase 是一个开源的 AI 应用工程平台，定位为 **Business-safe AI Runtime（业务安全 AI 运行时）**。它回答一个核心问题：**当 AI 不再只是聊天、而是真正操作企业的数据时，如何保证它越权不做、写了能查、错了能撤?**

平台提供一条完整链路：

```
新业务 / 既有系统 → 应用协议 → AI 生成业务模块 → 业务安全 Agent
    → 权限治理 → 人工确认 → 审计 → 可撤销 → 私有化部署
```

- **Build（开发期 AI）**：以「应用协议」为约定，由 AI 生成完整业务模块（实体 / CRUD / 权限 / AI 工具 / 审计），生成的是**可读、可改、可审计的源码**，而非低代码引擎。
- **Run（运行时 AI）**：AI Agent 的工具调用限定在用户数据范围内，写操作必须经人工确认，全部动作进入篡改即断链的审计哈希链（应用边界内；完整性可离线验证）。
- **Trust / Private Deploy**：数据主权——支持 Docker 单容器、离线、本地模型部署，AI 全程可审计；本系统内创建的写副作用可撤销，外部系统副作用如实呈现补偿状态。

> 三行代码之外的承诺：**AI 可以行动，但只在明确的业务边界内行动。**（*AI can act — but only within explicit business boundaries.*）

---

## 2. 项目背景与痛点

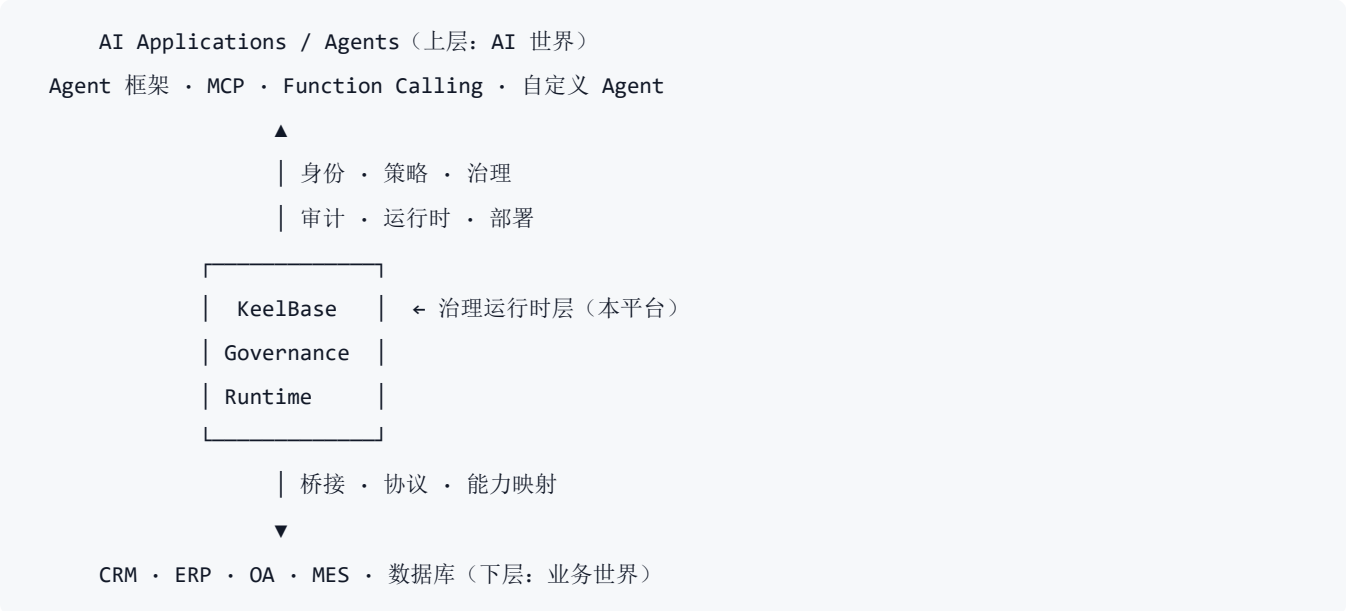
大模型能力爆发后，企业面临一个尴尬局面：

| 现状 | 问题 |
| --- | --- |
| 通用 Agent 框架（仅限对话/检索） | AI「只说不做」，无法真正帮业务人员完成工作 |
| 让 AI 直接调用企业 API | **越权风险**——AI 可能读取无权限数据、执行未授权写操作 |
| 黑盒决策 | 无法回答「AI 为什么这么做」，无法向监管证明合规 |
| 数据出境顾虑 | 云端大模型处理敏感业务数据，数据主权无法保证 |

企业的真实诉求不是「一个能聊天的机器人」，而是 **「一个受控、可审计、可纠错的业务员工」**。KeelBase 正是把 AI 从「对话工具」变成「受治理的业务执行者」的那层治理运行时底座。

---

## 3. 项目定位




- **向上（北向）**：任何 Agent 都能进入治理。通过开放标准（MCP / OpenAPI / Function Calling）接入，不重复造轮子。
- **向下（南向）**：任何既有系统都能获得 AI 能力。通过 Bridge（协议 + 能力映射）接入，**不推倒重来**。
- **中间**：治理运行时层——身份、策略、权限、人工确认、副作用控制、审计与撤销（本系统副作用）、私有化部署。

KeelBase 同时是完整的应用开发平台（从协议生成全栈业务应用），也是存量系统的 AI 治理运行时（Business-safe AI Runtime）——**两者共用同一个运行时。**

---

## 4. 技术架构

### 4.1 总体架构




平台采用「核心与 UI 无关、前后端分离、多端宿主」的架构：

| 层 | 技术 | 说明 |
| --- | --- | --- |
| 移动端 | Flutter 3.x + Material 3 | iOS / Android / Web 三端 |
| Web 宿主 | Vue3 + Element Plus（Vite + Pinia） | 工作台 + 管理台同一壳；React 19 + MUI 预览版 |
| 后端 | NestJS 11 + TypeScript | 模块化、装饰器驱动 |
| ORM | TypeORM | SQLite（开发）/ PostgreSQL（生产） |
| 认证授权 | JWT（access+refresh 轮换）+ CASL 行级权限 | 登录锁定 / Token 轮换 / 防枚举 / 防时序 |
| AI | OpenAI 兼容（DeepSeek / Qwen / 本地 Ollama） | Tool Calling / RAG / 记忆 / 子 Agent / 联网搜索 |
| 可观测性 | pino + Prometheus + OpenTelemetry | 结构化日志 / 指标 / 链路追踪 |
| 部署 | Docker + Nginx | 单容器 / 离线 / HTTPS |

### 4.2 业务安全 AI 运行时（核心模块）

AI 的每一次工具调用都经过统一治理层：

```
用户请求 → AI 理解 → 业务数据 → 工具调用
        → 权限校验 → 人工确认 → 副作用 → 审计 → 撤销
```

| 能力 | 说明 |
| --- | --- |
| 用户数据范围限定 | 每个工具调用携带认证用户，AI 只能触碰该用户的数据 |
| 人工确认 | 写操作执行前必须获得人类明确批准（批准 / 拒绝 / 本轮信任） |

| 能力 | 说明 |
| --- | --- |
| 篡改即断链审计（应用边界内） | 全部动作写入 SHA-256 审计哈希链，`/audit/verify` 可验证完整性；绕过系统正式写路径的改动即断链 |
| 副作用可撤销 | AI 创建的记录（事件 / 待办 / CRM 任务）可撤销，软删可经回收站恢复 |
| 可解释决策 | 「AI 为什么这么做」由决策轨迹回答（决策依据 / 权限依据 / 确认记录 / 审计） |
| 安全加固 | 提示注入防护、工具治理、SSRF 防护、AI 评测集 |

### 4.3 三旗舰 Reference Applications

三旗舰不是「三个 CRUD Demo」，而是三类 AI 工作方式的 Reference：

| | AI CRM | AI Project | AI Approval |
| --- | --- | --- | --- |
| AI 角色 | **AI Operator**（替用户执行） | **AI Collaborator**（与用户协作） | **AI Decision Assistant**（辅助决策） |
| 场景 | 找风险客户 → 建跟进任务 | 判断延期风险 → 建议推进 | 按政策预审 → 人工复核 |
| 读工具 | query_customers / analyze_customer_risk | query_projects / analyze_project_risk | query_approval_policies / review |
| 写工具 | create_followup_task（确认+可撤销） | create_project_task（确认+可撤销） | submit_approval_request（确认+可撤销） |
| 治理深度 | 风险打分（逾期/金额/未解决） | 风险打分（逾期任务/里程碑） | **AI 预审分级 + 人工复核** |

## 5. 核心功能

### 5.1 AI 真正完成业务工作（Run）

以旗舰 AI CRM 的黄金流程为例（演示主线）：

```
用户：「哪些客户本周最值得跟进？」

KeelBase AI：
  3 个客户需要关注…
    → 读取该客户的订单与跟进记录（数据范围限定在本人客户）
    → 按风险规则打分（逾期订单 / 高价值 / 未解决风险）
    → 「云帆商贸有逾期订单且连续两月未续约，建议跟进」
```

```
→ 询问「要我为云帆创建跟进任务吗？」
→ 用户确认 → 创建跟进任务 → 写入 CRM
→ 记录审计 → 允许撤销
```

一次演示胜过二十个功能清单——**AI 在调用者的数据范围内读数据 → 判断风险 → 高风险写在你确认后执行 → 动作可审计、可撤销。**

## 5.2 AI 应用工程（Build）

`keelbase init` 以自然语言 / SQL Schema / OpenAPI 为输入，生成完整业务模块：

```
自然语言 → 模块规格 → 应用协议 → 应用代码 → AI 工具 → 治理
```

生成产物是**普通源码**（NestJS 模块 + 前端页面 + 权限 + AI 工具 + 审计接线），无专有运行时元数据、无拖拽绑定——企业完全拥有产物。从零构建一个带 AI 的 CRM 模块约 **30 分钟。**

## 5.3 存量系统 AI 化（Existing System Alization）

不替换既有系统（CRM / ERP / OA / 十年 Java 老系统），通过 Bridge 让 AI 在治理约束下读取 / 写回：

```
既有 DB / OpenAPI / Java 系统 → 应用协议 → 生成模块 → AI 工具 + 治理 → 业务 Agent
```

## 5.4 治理与审计（Trust）

- **权限**：CASL 行级权限——AI 的访问限于调用者的数据范围（本人数据；组织作用域的资源按同组织可见），逐例断言见越权测试矩阵
- **确认**：写操作必须人工批准（Golden Flow e2e 验证）
- **审计**：操作审计 + AI 审计 + 审计哈希链（篡改检测），管理台可视化
- **可解释**：Explainable Authz——「为何某用户/某 AI 操作被允许或被拒」可反查依据
- **评测**：内置安全评测集（越权 / PII / 提示注入 / 写拒绝）

## 5.5 私有化部署（Private Deploy）

```
云端大模型 或 本地模型（Ollama）
    → 本地 Embedding → 本地 RAG → 业务安全 Agent → 本地审计
```

Docker 单容器一键启动（`docker run` 一条命令起全栈）、离线 / 内网部署、本地模型与向量化——**数据不出域。**

## 6. 应用场景

| 场景 | 说明 |
| --- | --- |
| 从零构建企业 AI 应用 | 以协议生成业务模块，30 分钟产出带治理的 AI 应用，适合内部工具 / 行业应用 / 系统集成商二次开发 |
| 存量业务系统 AI 化 | 给十年历史的 CRM / ERP / OA 添加 AI 能力，不推倒重来，AI 在既有数据上做风险分析、跟进、审批 |
| 受监管行业的数据主权 | 金融 / 政务 / 医疗等数据敏感场景，本地模型 + 私有化部署，数据不出域 |
| AI 应用的合规证明 | 审计哈希链 + 可解释授权 + 决策轨迹，向监管证明「AI 做了什么、为什么允许」 |
| Agent 生态的治理层 | 作为 Agent 框架（MCP / Function Calling）下方的治理运行时层，为任意 Agent 提供身份、策略、审计 |

## 7. 创新点

1. **业务安全的 Agent Harness（核心创新）**：不是「给 AI 加一层权限检查」，而是把**用户数据范围、人工确认、副作用、审计、撤销**作为 AI 工具调用的固有环节——AI 在执行层面被约束，而非事后追责。
2. **AI 按系统约定生成，而非内建生成器**：平台只提供「应用协议」约定，AI 负责生成真实源码；不锁定、无黑盒、产物完全归企业所有。
3. **不替换存量系统**：以 Bridge 连接既有技术栈，AI 在治理约束下与老系统协同，降低企业落地门槛。
4. **可解释授权（Explainable Authz）**：授权决策带依据（角色 + 资源范围 + 行级条件），管理台可排查「为何某用户被拒」，配合决策轨迹实现全链路可解释。
5. **篡改即断链的审计哈希链（应用边界内）**：AI 与操作审计链式哈希，篡改即被检测——为「AI 干了什么」提供可验证的完整性证据。
6. **AI 预审 + 人工复核**：审批场景下低风险自动通过、高风险转人工，治理与效率的平衡。
7. **三端一致 + 一套后端**：移动（Flutter）/ 小程序（Taro）/ Web 宿主（Vue3）共用同一后端，业务逻辑单点维护。

## 8. 工程质量

| 项 | 数据 |
| --- | --- |
| 后端测试 | 单元 + E2E 全量，覆盖率门槛 statements ≥86% / branches ≥72% / functions ≥81%（含安全模块专项门控） |
| 安全验证 | 权限矩阵 · 工具治理逐例断言 · 审计哈希链篡改检测 · 安全评测集 · 170+ 安全相关测试 |
| CI | GitHub Actions 33 项流水线（按 `ci.yml` 的 job 计）：lint / 构建 / 单元 / E2E / postgres 迁移一致性 / 架构边界 / 供应链检查 / Flutter 分析 |
| 代码规范 | ESLint + 架构边界门禁（Core 无 UI 框架依赖） |
| 版本发布 | SemVer 语义化，1.1.0 已发布（2026-10-05），CHANGELOG 全程维护 |
| 文档 | 中英双语：90+ 篇规格 / 手册 / 教程，README 一键体验 |

**可运行性验证**：`docker run` 一条命令启动全栈（后端 + 工作台 + 管理台 + 移动预览），内置演示账号，开箱即达完整黄金流程。

---

## 9. 开源与合规

- **开源许可**：Apache-2.0
- **仓库**：GitHub `rain6fish/KeelBase`（镜像 Gitee），公开可访问
- **原创性**：全部代码 / 文档 / 架构为本项目原创开发
- **合规**：依赖均采用开源许可（MIT / Apache-2.0 / ISC / BSD / OFL 为主），无闭源组件风险
- **版权自查（2026-08-29）**：第三方组件声明见 THIRD_PARTY_NOTICES.md——图像处理 sharp/libvips 为 LGPL-3.0 **独立库未修改使用**（可替换），jszip 按 MIT 选项使用；字体（Noto Sans SC / mdi / fontsource）为 OFL 开源字体；应用图标为项目原创/模板默认素材，演示视频为原创录制，无未授权第三方素材

---

## 10. 功能完成度说明

> 如实声明：以下为本项目相对设计文档（`docs/` 下全部 Spec）的功能完成度。

**已完成（约 95%）**：认证体系（JWT 轮换 / MFA / 登录锁定 / SSO OIDC / 邮箱验证 / 短信）、用户与权限（CASL 行级 / Explainable Authz）、事件 / 待办 / 通知 / 搜索、AI 助手（对话 / 工具 / RAG / 记忆 / 评测 / 副作用撤销 / MCP 出口 / Headless API）、三旗舰（CRM / PM / Approval）、组织架构、工作流引擎（FLOW）、插件 / Webhook / 数据导入 / 表单构建 / 模板市场、监控 / 审计 / 回收站 / 可观测性、Web 工作台 + 管理台（Vue3）、移动端（Flutter / Taro）、私有化部署（单容器 / 离线 / 本地模型）。

**未完成 / 部分完成（约 5%，详见相应 Spec 的「后续」章节）**：

- 推送服务部分驱动（极光已接入；FCM / APNs 预留）
- 部分第三方 OAuth 提供商（微信 / 支付宝 / OIDC 已实现；部分国际提供商预留）
- 组织模块部分高级能力（部门树只读查询等按迭代推进）
- 微信小程序部分能力（订阅消息真机联调待 AppID/Secret 配置）
- 其余以 `docs/` 各 Spec 标注为准

> 上述未完成项不影响核心链路（Build → Run → Trust → Private Deploy）的完整闭环，且持续以迭代方式交付。

---

## 11. 演示

- **一键体验**：`docker run -d --name keelbase -p 3000:3000 ghcr.io/rain6fish/keelbase:latest`
- **演示账号**：`alex / Alex@2026$Demo`（工作台）· `admin / Admin@2026$KeelBase`（管理台）——所有环境统一
- **黄金流程**：登录 → AI CRM → 「哪些客户本周最值得跟进？」→ 观察 AI 分析 → 确认跟进任务 → 查看审计轨迹

---

*KeelBase — Build and Run Business-safe AI Applications.*